

Filters in ASP.NET Core MVC

1. What are Filters in ASP.NET Core MVC?

Filters are attributes you can apply to controllers or actions to execute code before or after specific stages in the request pipeline (e.g., authorization, action execution, result generation).

2. List the main types of Filters in ASP.NET Core MVC.

1. Authorization Filter ([IAuthorizationFilter](#))
2. Resource Filter ([IResourceFilter](#))
3. Action Filter ([IActionFilter](#))
4. Exception Filter ([IExceptionHandler](#))
5. Result Filter ([IResultFilter](#))

3. What is the execution order of filters in ASP.NET Core MVC?

1. Authorization → 2. Resource → 3. Action → 4. Exception → 5. Result

4. How to implement a custom Action Filter in ASP.NET Core MVC?

```
public class LogActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        Console.WriteLine("Action executing...");
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        Console.WriteLine("Action executed.");
    }
}
```

Register by adding [\[LogActionFilter\]](#) on Controller or Action method.

5. Difference between [IActionFilter](#) and [IAsyncActionFilter](#):

- [IActionFilter](#): Synchronous methods
 - [IAsyncActionFilter](#): Asynchronous method ([OnActionExecutionAsync](#)) for non-blocking operations.
-

Middleware in ASP.NET Core

1. What is Middleware in ASP.NET Core?

Middleware handles requests and responses in the HTTP pipeline and is configured in [Program.cs](#).

2. How to register custom middleware in Program.cs (ASP.NET Core 6+)?

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
  
app.UseMiddleware<RequestLoggingMiddleware>();  
  
app.MapControllers();  
app.Run();
```

3. Why call `await _next(context)` in middleware?

To pass the HTTP context to the next middleware component in the pipeline.

4. How to implement custom middleware?

```
public class RequestLoggingMiddleware  
{  
    private readonly RequestDelegate _next;  
  
    public RequestLoggingMiddleware(RequestDelegate next)  
    {  
        _next = next;  
    }  
  
    public async Task InvokeAsync(HttpContext context)  
    {  
        Console.WriteLine($"Request Path: {context.Request.Path}");  
        await _next(context); // Continue pipeline  
    }  
}
```

Common Interview Questions – Filters & Middleware in ASP.NET Core

1. What is the difference between Middleware and Filters in ASP.NET Core?

- **Middleware** is applied globally to all HTTP requests and responses in the pipeline. It handles cross-cutting concerns like authentication, logging, CORS, etc.
- **Filters** are applied at the MVC level to controllers or actions and are useful for logic like validation, authorization, and result manipulation specific to MVC processing.

2. What are the different types of Filters in ASP.NET Core MVC?

1. **Authorization Filter (IAuthorizationFilter)** – Runs first, used for permission checks.
 2. **Resource Filter (IResourceFilter)** – Runs before model binding and after action execution, used for caching or resource setup.
 3. **Action Filter (IActionFilter)** – Runs before and after the action method executes.
 4. **Exception Filter (IExceptionHandler)** – Catches unhandled exceptions in controller actions.
 5. **Result Filter (IResultFilter)** – Runs before and after the result (e.g., View or JSON result) is generated.
-

3. What is Middleware in ASP.NET Core and why is it used?

Middleware is software components in the HTTP request pipeline that handle HTTP requests and responses.

Common uses:

- Logging
- Authentication
- Exception handling
- CORS configuration
- Static file serving

4. Where do you configure Middleware in ASP.NET Core 6 or later?

Middleware is configured in `Program.cs` using the `app.Use...` methods. Example:

```
var builder = WebApplication.CreateBuilder(args);  
var app = builder.Build();  
  
app.UseMiddleware<RequestLoggingMiddleware>();  
app.UseRouting();  
app.UseAuthorization();  
app.MapControllers();  
  
app.Run();
```

5. What is the order of execution of filters in ASP.NET Core MVC?

1. Authorization Filter
2. Resource Filter
3. Action Filter
4. Exception Filter
5. Result Filter

This order ensures correct flow of request processing.

6. How do you implement a custom Middleware in ASP.NET Core?

```
public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;

    public RequestLoggingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        Console.WriteLine($"Request Path: {context.Request.Path}");
        await _next(context);
    }
}
```

Register in Program.cs:

```
app.UseMiddleware<RequestLoggingMiddleware>();
```

7. How do you implement a custom Action Filter in ASP.NET Core MVC?

```
public class LogActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        Console.WriteLine("Action executing...");
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        Console.WriteLine("Action executed.");
    }
}
```

Apply by adding `[LogActionFilter]` attribute to Controller or Action method.

8. When would you use Middleware instead of a Filter?

- Use Middleware for cross-cutting concerns like authentication, logging, CORS, exception handling at the global level.
- Use Filters when logic is specific to MVC (action-based validation, result formatting, etc.).

9. What is the significance of calling `await _next(context)` in custom Middleware?
`await _next(context)` passes control to the next middleware in the pipeline. Without it, the pipeline stops, and no further processing occurs.

10. Can you apply multiple Filters to a single action or controller? How?

Yes, by stacking multiple filter attributes:

```
[LogActionFilter]
```

```
[AuthorizeFilter]
```

```
public IActionResult MyAction() { ... }
```

Or register globally in Program.cs:

```
builder.Services.AddControllers(options =>
{
    options.Filters.Add(new LogActionFilter());
    options.Filters.Add(new AuthorizeFilter());
});
```

11. What is the purpose of the **OnActionExecuting** and **OnActionExecuted** methods in an Action Filter?

- **OnActionExecuting**: Runs **before** the action method execution (used for pre-processing logic like validation).
- **OnActionExecuted**: Runs **after** the action method execution (used for post-processing logic like logging).

12. What is the difference between synchronous and asynchronous Filters in ASP.NET Core?

- **Synchronous (IActionFilter)**: Runs synchronously using **OnActionExecuting** and **OnActionExecuted**.
 - **Asynchronous (IAsyncActionFilter)**: Runs asynchronously using **OnActionExecutionAsync**, which allows non-blocking operations (e.g., database calls).
-

13. How can you apply a Filter globally in ASP.NET Core MVC?

```
builder.Services.AddControllers(options =>
{
    options.Filters.Add(new LogActionFilter());
});
```

This applies the filter to all controller actions.

14. How do Exception Filters differ from Middleware exception handling?

- **Exception Filters** handle exceptions thrown by controller actions and can return specific views or results.
- **Middleware Exception Handling** handles all exceptions in the pipeline, including non-MVC endpoints, and is better for centralized logging and global error handling.

15. Can you short-circuit the request in Middleware or Filters? How?

Middleware Example:

```
public async Task InvokeAsync(HttpContext context)
{
    if (!context.Request.Headers.ContainsKey("X-Auth"))
    {
        context.Response.StatusCode = StatusCodes.Status401Unauthorized;
        await context.Response.WriteAsync("Unauthorized");
        return; // Stops further pipeline execution
    }

    await _next(context);
}
```

Filter Example:

```
public void OnActionExecuting(ActionExecutingContext context)
{
    if (!context.HttpContext.Request.Headers.ContainsKey("X-Auth"))
    {
        context.Result = new UnauthorizedResult(); // Stops action execution
    }
}
```